

Starting Node of a Loop in a Linked List

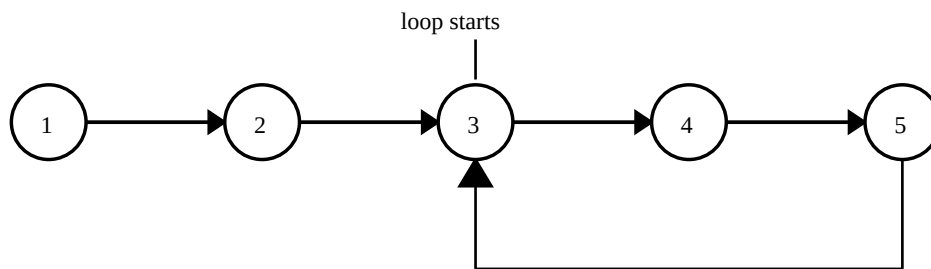
Personal DSA Notes

1. Introduction

Given a linked list, find the node where a loop begins. If there is no loop, return null. The answer must be the actual node/reference, not just its value.

2. Problem Statement

A list may be linear or cyclic. For a cyclic list, the first node that belongs to the repeated cycle is the starting node of the loop.



3. Approach 1 — Hashing

Traverse the list with a temporary pointer and store each complete node reference in a set. Before storing a node, check whether that exact node has already been seen. The first repeated node is the loop start.

Important: store the node, not its value. Different nodes can contain the same value.

```
temp = head
while temp != null:
    if temp is already in set:
        return temp
    add temp to set
    temp = temp.next
return null
```

Java

```
class Solution {
    public ListNode detectCycle(ListNode head) {
        Set<ListNode> seen = new HashSet<>();

        ListNode temp = head;

        while (temp != null) {
            if (seen.contains(temp)) {
                return temp;
            }

            seen.add(temp);
            temp = temp.next;
        }

        return null;
    }
}
```

Time: $O(N)$ expected. Extra space: $O(N)$.

4. Approach 2 — Tortoise and Hare

This method has two phases. First detect the loop. Then locate its starting node.

Phase 1: slow moves one step and fast moves two steps. If they collide, a loop exists. If fast reaches null, there is no loop.

```
slow = head
fast = head

while fast != null AND fast.next != null:
    slow = slow.next
    fast = fast.next.next

    if slow == fast:
        loop found
```

Phase 2: move slow back to head. Keep fast at the collision point. Move both one step at a time. Their next collision is the loop-start node.

```
slow = head

while slow != fast:
    slow = slow.next
    fast = fast.next

return slow
```

5. Complete Java Code

```
class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;

        // Phase 1: detect a loop
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;

            if (slow == fast) {
                break;
            }
        }

        // No loop
        if (fast == null || fast.next == null) {
            return null;
        }

        // Phase 2: find the starting node
        slow = head;

        while (slow != fast) {
            slow = slow.next;
            fast = fast.next;
        }

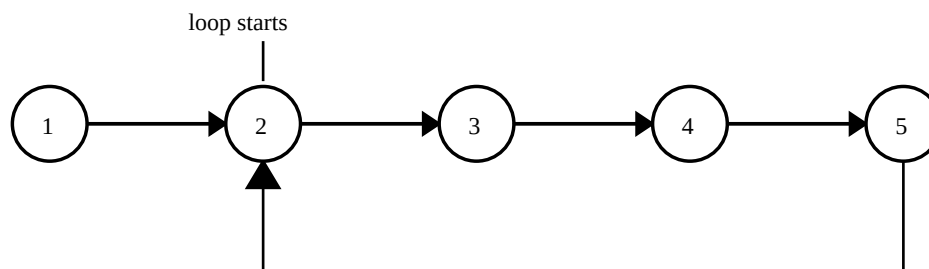
        return slow;
    }
}
```

6. Intuition / Proof

During phase 1, fast gains one node on slow during every iteration because fast moves two steps while slow moves one. If their gap is D , it becomes $D-1$, then $D-2$, and eventually 0. Therefore they must collide inside the loop.

After the collision, let $L1$ be the distance from the head to the loop start. Let D be the remaining distance around the loop from the collision point to the loop start. The pointer-distance relationship gives the loop length as $L1 + D$. Resetting one pointer to head and moving both pointers one step at a time makes the two distances line up at the loop start.

7. Visual — Collision and Reset



The first meeting proves that a loop exists. Resetting one pointer to head and moving both pointers at one step per iteration makes their second meeting identify the starting node.

8. Edge Cases

- Empty list $\rightarrow$ null
- One node pointing to itself $\rightarrow$ that node
- One node with next = null $\rightarrow$ null
- Linear list $\rightarrow$ null
- Loop starts at head $\rightarrow$ head
- Loop starts in the middle $\rightarrow$ exact loop-start node

9. Complexity

Tortoise and Hare: $O(N)$ time, $O(1)$ extra space.

Hashing: $O(N)$ expected time, $O(N)$ extra space.

10. Quick Revision

1. `slow = head, fast = head`
2. `slow += 1, fast += 2`
3. `fast reaches null -> return null`
4. `slow == fast -> loop exists`
5. `slow = head`
6. Move both by one
7. `slow == fast -> return that node`

11. Key Interview Point

Loop detection and loop-start finding are two different steps. Detecting a cycle is not enough; the second phase returns the exact node where the cycle begins.